

深圳大学实验报告

课程名称： 算法设计与分析

实验名称： 实验四 动态规划—鸡蛋掉落问题

学院： 计算机与软件学院 专业： 计算机科学与技术

报告人： 黄家成 学号： 2024150286 班级： 国际班

同组人： 黄瑜 2041500080

指导教师： 吴定明

实验时间：

实验报告提交时间：

一、 实验目的

1. 掌握动态规划算法设计思想。
2. 掌握鸡蛋坠落问题的动态规划解法。

二、 动态规划算法模型设计

动态规划解决鸡蛋掉落问题的核心思想是“寻找最优子结构”并“最小化最坏情况”

1. 状态定义

设 $dp[i][j]$ 表示在拥有 i 个鸡蛋、面对 j 层楼时，在最坏情况下确定门槛楼层所需的最少试验次数。

2. 边界条件

(1) 楼层数为 0 或 1 时，试验次数等于楼层数：

$$dp[i][0]=0, dp[i][1]=1$$

(2) 只有一个鸡蛋时，只能逐层尝试，最坏情况需要试满全部楼层：
层： $dp[1][j]=j$

3. 状态转移方程

假设我们在第 k 层抛下鸡蛋 ($1 \leq k \leq j$)，会有两种确定性的结果：

- (1) **鸡蛋碎了**：门槛在下方，鸡蛋减 1，剩余需测楼层为 $k-1$ ，后续代价为 $dp[i-1][k-1]$;
- (2) **鸡蛋没碎**：门槛在上方，鸡蛋不变，剩余需测楼层为 $j-k$ ，后续代价为 $dp[i][j-k]$ 。

因为无法预知鸡蛋是否会碎，我们必须考虑最坏情况，即取两者最大值。我们要遍历所有可能的抛掷楼层 k ，寻找能让这个最坏

情况最小化的抛掷点, 加上当前在第 k 层扔的这 1 次操作, 方程如下:

$$dp[i][j] = 1 + \min \{ \max(dp[i-1][k-1], dp[i][j-k]) \}$$
 (注: 其中 k 的取值范围为 $1 \leq k \leq j$)

4.核心伪代码

```
Function EggDrop_DP(e, f):
// e: 鸡蛋总数, f: 楼层总数
// 1. 初始化 DP 表格, 大小为 (e+1) x (f+1)
Create a 2D array dp[e+1][f+1] initialized to 0
// 2. 填充边界条件
// 2.1 楼层为 0 和 1 的情况      For i = 1 To e:
    dp[i][0] = 0
    dp[i][1] = 1
// 2.2 只有一个鸡蛋的情况
For j = 1 To f:
    dp[1][j] = j
// 3. 自底向上填表求解
// 外层循环: 遍历鸡蛋数从 2 到 e
For i = 2 To e:
    // 中层循环: 遍历楼层数从 2 到 f
    For j = 2 To f:
        dp[i][j] = ∞
        // 内层循环: 尝试在第 j 层楼模型下的每一个抛掷点 k
        For k = 1 To j:
            // 计算最坏情况下的试验次数
            worst_case = 1 + MAX(dp[i-1][k-1], dp[i][j-k])
            // 寻找最优策略 (最小化最坏情况)
            If worst_case < dp[i][j]:
                dp[i][j] = worst_case
// 4. 返回最终结果
Return dp[e][f]
```

5.理论复杂度分析

(1)**时间复杂度:** $O(e \cdot f^2)$ 。对于状态数量, DP 表格的大小为 $(e+1) \times (f+1)$, 因此总共有 $O(e \cdot f)$ 个状态需要计算; 对于状态转移代价, 在计算每一个特定的状态 $dp[i][j]$ 时, 我们需要执行一个从 1 到 j 的内

层 for 循环。在最坏情况下，即计算最高层 f 时，这个循环需要执行 f 次。状态数 $O(e \cdot f)$ 乘以转移代价 $O(f)$ ，总时间复杂度为 $O(e \cdot f^2)$

(2) **空间复杂度：** $O(e \cdot f)$ 。算法为了避免重复计算，开辟了一个二维数组 dp 来保存所有子问题的解。数组的行数为 $e+1$ ，列数为 $f+1$ 。因此，总共需要消耗 $O(e \cdot f)$ 的内存空间。

三、 利用蛮力法验证算法正确性

1. 算法思路简述

蛮力法的核心思想是直接将数学逻辑转化为递归过程，在鸡蛋掉落问题中，蛮力法的思路如下：

(1) 边界条件:

- a. 当楼层 $f = 0$ 或 $f = 1$ 时，无论多少个鸡蛋，所需的最少试验次数就是楼层数本身（0 次或 1 次）
- b. 当鸡蛋数 $e = 1$ 时，只能从第 1 层逐层往上测试，最坏情况下需要试 f 次，返回 f

(2) 递归逻辑: 如果鸡蛋数 $e > 1$ 且楼层数 $f > 1$ ，则尝试从第

1 层到第 f 层中的每一层（设为第 k 层）扔下鸡蛋

- a. 如果碎了，往下需要继续测 $(e-1, k-1)$
- b. 如果没碎，往上需要继续测 $(e, f-k)$
- c. 在这个 k 层测试的代价是： $1 + \max(\text{碎了的代价}, \text{没碎的代价})$
- d. 最后，在所有楼层 k 对应的代价中，取最小值返回

2. 蛮力法伪代码

```

Function EggDrop_BruteForce(e, f):
    // 1. 处理边界条件
    If f == 0 OR f == 1 Then
        Return f
    If e == 1 Then
        Return f
    min_drops = ∞
    // 2. 遍历当前情况下，所有可能的投掷楼层 k
    For k = 1 To f:
        // 取“碎”与“没碎”两种情况下的最大值（最坏情况）
        worst_case_drops = MAX( EggDrop_BruteForce(e - 1, k - 1),
                                EggDrop_BruteForce(e, f - k) )
        // 加上当前在第 k 层投掷的 1 次
        current_drops = 1 + worst_case_drops
    // 3. 取所有 k 中能让最坏情况最小化的值
    If current_drops < min_drops Then
        min_drops = current_drops
    Return min_drops

```

3. 复杂度分析

- (1) **时间复杂度**：指数级 $O(2^f)$ 或更高。因为算法没有使用任何数组来记录已经计算过的子问题。例如，计算第 5 层时分裂出的子问题，在计算第 6 层时又会被重新分裂计算一次。这导致递归树呈指数级爆炸式增长。
- (2) **空间复杂度**： $O(f)$ 。空间复杂度主要取决于递归调用栈的深度。在最坏情况下，递归栈的深度等于楼层数 f 。

4. 实验代码运行结果

```

[Test 1] 生成数据 -> 鸡蛋数(e): 3, 楼层数(f): 13
- 蛮力法结果: 4 | 耗时: 0.0136 秒
- DP算法结果: 4 | 耗时: 0.0010 秒

```

```
[Test 2] 生成数据 -> 鸡蛋数(e): 2, 楼层数(f): 15
- 蛮力法结果: 5 | 耗时: 0.0091 秒
- DP算法结果: 5 | 耗时: 0.0000 秒
```

```
[Test 3] 生成数据 -> 鸡蛋数(e): 2, 楼层数(f): 7
- 蛮力法结果: 4 | 耗时: 0.0000 秒
- DP算法结果: 4 | 耗时: 0.0000 秒
```

```
[Test 4] 生成数据 -> 鸡蛋数(e): 3, 楼层数(f): 5
- 蛮力法结果: 3 | 耗时: 0.0000 秒
- DP算法结果: 3 | 耗时: 0.0000 秒
```

```
[Test 5] 生成数据 -> 鸡蛋数(e): 2, 楼层数(f): 5
- 蛮力法结果: 3 | 耗时: 0.0000 秒
- DP算法结果: 3 | 耗时: 0.0000 秒
```

实验代码进行了五次测试，都是随机产生 f , e 的值，然后对小数据模型利用蛮力法测试算法的正确性，以上五次测试结果，经验证与动态规划算法的结果是一致的

四、动态规划算法效率测试与规模分析

1.测试目标：验证基础动态规划算法的时间效率，观察其耗时随楼层数 f 和鸡蛋数 e 增加的增长趋势，并与理论时间复杂度进行比对。

2. 测试方法：

- 有限时间规定为 30 秒左右，划分不同的数据规模等级

小型数据范围：鸡蛋数 e 在 $[3, 5]$ 之间，楼层数 f 在 $[50, 100]$ 之间；

中型数据范围：鸡蛋数 e 在 $[5, 8]$ 之间，楼层数 f 在 $[400, 600]$ 之间；

大型数据范围：鸡蛋数 e 在 $[8, 12]$ 之间，楼层数 f 在 $[1000, 1200]$

之间;

极限型数据范围: 鸡蛋数 e 在 $[10, 15]$ 之间, 楼层数 f 在 $[4500, 5500]$ 之间

- 在每个规模下, 随机生成 e 和 f 的值;
- 利用 Python 的 `time.perf_counter()` 高精度计时器测量算法执行时间;

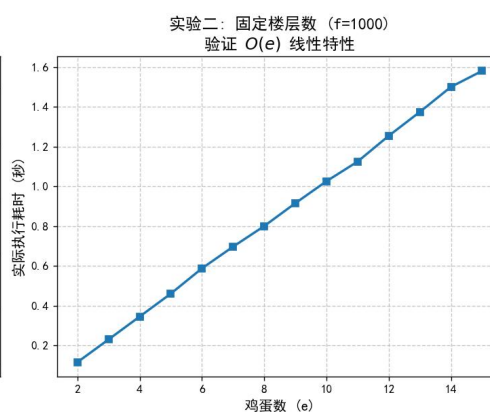
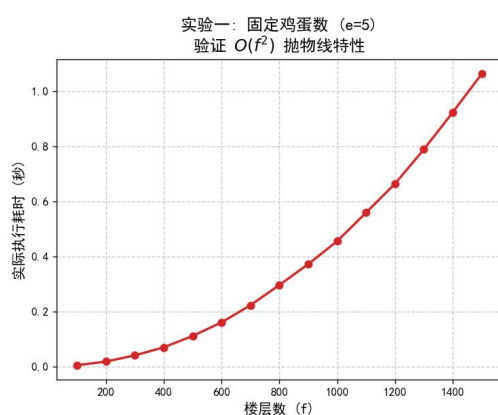
3.代码运行结果

(1) 随机测试的结果

规模等级	鸡蛋数(e)	楼层数(f)	最少试验次数	耗时(秒)
小型	3	76	8	0.0013
中型	5	550	10	0.1329
大型	9	1074	11	1.0706
极限型	15	4757	13	37.1596

规模等级	鸡蛋数(e)	楼层数(f)	最少试验次数	耗时(秒)
小型	5	73	7	0.0023
中型	8	440	9	0.1473
大型	8	1112	11	1.0128
极限型	15	4847	13	38.8524

(2) 为验证理论时间复杂度 $O(e \cdot f^2)$, 分别固定 e 和 f 进行了两组实验, 分析可知实际效率的曲线是符合理论效率的



(3) 算法能处理的数据最大规模推导

根据极限型规模的本机实测数据（鸡蛋数 $e=15$ ，楼层数 $f=4847$ ，实际耗时 38.8524 秒），结合基础动态规划算法耗时与 $O(e \cdot f^2)$ 成正比的理论特性，进行逆向降维推算：

- 目标时间 30 秒占实际耗时的比例约为 0.772
- 在 $e=15$ 保持不变的情况下，由于 $O(f^2)$ 的平方级特性，最大处理楼层数应为当前的 $\sqrt{0.772}$ 倍（约 0.878 倍）。计算得出极限楼层约为 $4847 \times 0.878 \approx 4255$ 层。
- 若将鸡蛋数降低，则可将节省出的线性算力转移给楼层计算。例如 e 降至 10（释放 1.5 倍算力），极限楼层可扩大至当前的 $\sqrt{(0.772 \times 1.5)} \approx 1.076$ 倍，即 5215 层。

五、对原算法的优化

1. 空间效率优化

(1) 当前瓶颈：基础算法开辟了一个大小为 $(e+1) \times (f+1)$ 的二维数组，空间复杂度为 $O(e \cdot f)$ 。当楼层 f 达到十万级时，会占用庞大的内存，甚至引发内存溢出

(2) 优化方案：观察状态转移方程 $dp[i][j] = 1 + \max(dp[i-1][k-1], dp[i][j-k])$ 可以发现，计算第 i 个鸡蛋的状态时，仅仅依赖于“当前行（第 i 行）”和“上一行（第 $i-1$ 行）”的数据，更早的数据完全不需要保留

(3) 优化结果：我们可以采用“滚动数组”技术，只维护两个长度

为 $f+1$ 的一维数组（分别代表当前鸡蛋和上一个鸡蛋的状态）交替更新。这能将空间复杂度从 $O(e \cdot f)$ 降至 $O(f)$

2. 时间效率优化方案

（1）当前瓶颈：基础算法的时间复杂度为 $O(e \cdot f^2)$ ，最耗时的是最内层遍历抛掷楼层 k 的 for 循环，时间复杂度为 $O(f)$

（2）优化方案：在状态转移时，我们要寻找 $1 + \max(dp[i-1][k-1], dp[i][j-k])$ 的最小值。随着抛掷楼层 k 的增加：

a. 鸡蛋碎了的代价 $dp[i-1][k-1]$ 是一个随 k 单调递增的函数。

b. 鸡蛋没碎的代价 $dp[i][j-k]$ 是一个随 k 单调递减的函数。

我们要找这两个函数最大值的“最低点”，其实就是这两个单调函数图像的交点附近。既然具有单调性，我们完全不需要从 1 到 j 挨个遍历 k ，而是可以在内层循环直接使用二分查找法来寻找这个最优的抛掷点 k

（3）优化结果：将最内层 $O(f)$ 的线性扫描优化为 $O(\log f)$ 的二分查找。总时间复杂度可从 $O(e \cdot f^2)$ 降至 $O(e \cdot f \cdot \log f)$

3. 优化后代码运行结果

选择和原算法相同的 e 与 f ，输出结果一致，耗时也明显减少

```
=====
极限规模数据优化测试 -> 鸡蛋数(e): 15, 楼层数(f): 4847
=====

计算得出的最少试验次数 : 13
优化后实际执行耗时      : 0.198502 秒

原基础算法耗时          : 38.8524 秒
=====

进程已结束，退出代码为 0
```

六、与 AI 的交互过程

1.使用的大语言模型： Gemini 3.1pro

2.重点交互过程

（1）将课件输入给 ai，通过提问的方式使用 ai 学习动态规划的算法思想，同时利用 ai 工具再次熟悉 Python 中的画图工具以及计算算法具体时间工具的使用

（2）最开始利用比值法来验证理论效率是否正确，但经过计算发现实际耗时扩大了 8.06 倍，而理论推算只有 6.69 倍。接着通过对 ai 进行提问，ai 给出了“大 O 记号忽略的常数项”以及底层硬件的“Cache Miss（缓存未命中）”等可能原因。后面我们才重新想出采用控制变量的办法，进行对理论效率的验证

（3）起初，我们尝试了一些比较保守的优化手段，如在进入 j 循环之前，提前把“上一行”和“当前行”的引用提取为局部变量，通过减少指针寻址次数，，试图挤出一点硬件性能来对原算法进行优化。ai 提示利用函数的单调递增和递减特性，直接用二分查找取代了内层的遍历操作

指导教师批阅意见：

成绩评定：

指导教师签字：
年 月 日

备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。